

Comparing Containers

History

[P0805R0](#) was presented to LEWG in Albuquerque, and was received favorably. They asked for an R1 that also included the rest of the sequence containers (`<deque>`, `<list>`, `<forward_list>` and `<vector>`) and forwarded the paper to LWG.

Rationale

When I was attempting to implement [P0122R5](#), I became frustrated with the inability to compare spans of different lengths, and sometimes spans of the same lengths as well. I started thinking about this in general, and realized that our comparison conventions for containers are limited, but for no good reason.

In C++17, `std::optional` has a nice set of comparison operators.

- An `optional<T>` can be compared to a `T`.
- An `optional<T>` can be compared to a `U`.
- An `optional<T>` can be compared to an `optional<T>`.
- An `optional<T>` can be compared to an `optional<U>`.
- An `optional<T>` can be compared to a `nullopt_t`.

But an `array<int, 5>` cannot be compared to an `array<int, 6>`. Nor can an `array<int, 5>` cannot be compared to an `array<long, 5>`. This seems like an artificial limitation to me - these comparisons "make sense", we just don't implement them. We *can* compare a `vector<int>` that contains five elements to a `vector<int>` that contains six elements, but not a `vector<int>` to a `vector<long>`, no matter what size. And if the allocators are different, that won't work either.

As people write more and more generic code, these arbitrary limitations will cause increasing friction.

In the header `<algorithm>` we have `lexicographical_compare`. This is a very general algorithm - it compares two sequences, which may be of differing lengths, and/or different underlying types, and returns whether one is "less than" the other. But that generality is not reflected in the containers, even though their comparisons are defined in terms of `lexicographical_compare` (Table 85).

The same logic applies to `std::tuple`. The comparison operators for `tuple` require that both tuples be the same size ([`tuple.rel`]/1) but (surprisingly) not that they contain the same types.

Note: I have not suggested changes to `basic_string`, because it does not use `lexicographical_compare`, but delegates to a traits class.

Suggested changes

These wording changes are relative to [N4713](#).

In [`tuple.rel`]/1, change:

Requires: For all i , where $0 \leq i$ and $i < \min(\text{sizeof}...(TTypes), \text{sizeof}...(UTypes))$, ~~$\text{sizeof}...(TTypes)$~~ , $\text{get}<i>(t) == \text{get}<i>(u)$ is a valid expression returning a type that is convertible to `bool`. ~~$\text{sizeof}...(TTypes) == \text{sizeof}...(UTypes)$~~ ;

In [tuple.rel]/2, change:

Returns: false if $\text{sizeof}...(TTypes) \neq \text{sizeof}...(UTypes)$, otherwise, true if $\text{get}<i>(t) == \text{get}<i>(u)$ for all i , otherwise false . For any two zero-length tuples e and f , $e == f$ returns true .

In [tuple.rel]/4, change:

Requires: For all i , where $0 \leq i$ and $i < \min(\text{sizeof}...(TTypes), \text{sizeof}...(UTypes))$, ~~$i < \text{sizeof}...(TTypes)$~~ , both $\text{get}<i>(t) < \text{get}<i>(u)$ and $\text{get}<i>(u) < \text{get}<i>(t)$ are valid expressions returning types that are convertible to `bool`. ~~$\text{sizeof}...(TTypes) == \text{sizeof}...(UTypes)$~~ ;

In [tuple.rel]/5, change:

Returns: The result of a lexicographical comparison between t and u . A zero-length tuple is lexicographically less than any non-zero-length tuple. The result is defined as: $(\text{bool})(\text{get}<0>(t) < \text{get}<0>(u)) \ || \ (!(\text{bool})(\text{get}<0>(u) < \text{get}<0>(t)) \ \&\& \ \text{rtail} < \text{utail})$, where rtail for some tuple r is a tuple containing all but the first element of r . For any two zero-length tuples e and f , $e < f$ returns false .

In [array.syn], add the following functions:

```
template <class T1, class T2, size_t N1, size_t N2>
    bool operator==(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
    bool operator!=(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
    bool operator<(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
    bool operator>(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
    bool operator<=(const array<T1,N1>& x, const array<T2,N2>& y);
template <class T1, class T2, size_t N1, size_t N2>
    bool operator>=(const array<T1,N1>& x, const array<T2,N2>& y);
```

In [deque.syn], add the following functions:

```
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator==(const deque<T1, Allocator1>& x, const deque<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator<(const deque<T1, Allocator1>& x, const deque<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator!=(const deque<T1, Allocator1>& x, const deque<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator>(const deque<T1, Allocator1>& x, const deque<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator>=(const deque<T1, Allocator1>& x, const deque<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator<=(const deque<T1, Allocator1>& x, const deque<T2, Allocator2>& y);
```

In [forward_list.syn], add the following functions:

```
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator==(const forward_list<T1, Allocator1>& x, const forward_list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator<(const forward_list<T1, Allocator1>& x, const forward_list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
```

```

    bool operator!=(const forward_list<T1, Allocator1>& x, const forward_list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator> (const forward_list<T1, Allocator1>& x, const forward_list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator>=(const forward_list<T1, Allocator1>& x, const forward_list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator<=(const forward_list<T1, Allocator1>& x, const forward_list<T2, Allocator2>& y);

```

In [list.syn], add the following functions:

```

template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator==(const list<T1, Allocator1>& x, const list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator< (const list<T1, Allocator1>& x, const list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator!=(const list<T1, Allocator1>& x, const list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator> (const list<T1, Allocator1>& x, const list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator>=(const list<T1, Allocator1>& x, const list<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator<=(const list<T1, Allocator1>& x, const list<T2, Allocator2>& y);

```

In [vector.syn], add the following functions:

```

template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator==(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator< (const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator!=(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator> (const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator>=(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);
template<class T1, class T2, class Allocator1, class Allocator2>
    bool operator<=(const vector<T1, Allocator1>& x, const vector<T2, Allocator2>& y);

```

Implementation status

I have implemented the changes for both [tuple](#) and [array](#).

Future Directions

- Provide compatibility with [P0515 - Consistent Comparisons](#).
- Research and decide if similar functionality should be provided for the associative containers. P0809R0 - "Comparing Unordered Containers" (no link yet) may influence this direction.